PROJETO: NutriVibe

Curso: Engenharia da Computação

Disciplina: PI: Desenvolvimento de Sistemas Web **Código:** 218040

Turma: a confirmar

Professor: Fernando Luiz de Almeida Silveira

Nome do Grupo: NutriVibe

Paulo Capreti — RA: 26028858

Davi Pioli — RA: 26005199

Victor Almeida — RA: 26003836

Marcelo Dalbone — RA: 25001986

Akiane Merbach — RA: 26027003

Campinas, 2º semestre de 2026

1. Ideação do Projeto

O NutriVibe aborda a dificuldade de organizar registros alimentares e ampliar a variedade da alimentação respeitando restrições e seletividade. O público-alvo previsto é composto por responsáveis e crianças, especialmente famílias que encontram dificuldades relacionadas à aceitação de cores, texturas e alimentos. A solução atua no contexto cotidiano das refeições e da educação alimentar, sem substituir acompanhamento profissional.

A proposta possui relevância social ao facilitar o acesso a informações e à organização da rotina. Relaciona-se ao ODS 3 — Saúde e Bem-Estar — pelo apoio educativo, e ao ODS 12 — Consumo e Produção Responsáveis — pelos conteúdos sobre planejamento e redução de desperdício. Esses são impactos pretendidos; não foram fornecidas evidências de atendimento comunitário ou de mudança de hábitos já mensurada.

- A solução reúne dois sites: uma landing page para apresentação e cadastro consentido de contatos e um aplicativo para perfil, preferências, registro alimentar, resumo diário e conteúdos educativos. O usuário pode selecionar um alimento ou enviar uma foto, confirmar a identificação e informar preparo e peso para obter uma estimativa nutricional baseada na TACO.
- A viabilidade dentro do semestre decorre do uso de HTML, CSS e JavaScript, de um catálogo delimitado e da separação entre regras, interface e armazenamento. O projeto foi iniciado em agosto, com a implementação do aplicativo concentrada em outubro, conforme informado pela equipe. O valor entregue é organizar informações e apoiar escolhas compatíveis com o perfil; a proposta não realiza diagnóstico ou prescrição alimentar.
-
-
-
-
-
-
-
-

2. Protótipo da Solução

A representação visual evoluiu para páginas navegáveis. A landing page apresenta problema, solução, funcionalidades, recursos planejados e formulário de contato. O aplicativo possui as telas Visão geral, Diário alimentar, Meu dia, Preferências e Aprender. A versão publicada permite consultar diretamente essas interfaces.

O fluxo básico começa pelas preferências, segue pelo diário e termina no resumo do dia. No perfil, o usuário informa nome, restrições e seletividade. No diário, escolhe data e refeição e seleciona alimento pelo catálogo ou por foto. Ao confirmar

uma referência nutricional, informa o peso. O resumo reúne grupos alimentares, nutrientes disponíveis e opções compatíveis com o perfil.

A identidade visual utiliza a marca NutriVibe em verde e amarelo. A landing page mantém uma composição de apresentação com cores sólidas. O aplicativo adota aparência de painel, com navegação lateral no computador, indicadores e registros do dia. Em telas menores, utiliza menu e organização responsiva. A marca possui versão com transparência, sem as grandes margens do arquivo original.

- A solução foi pensada para ambiente Web responsivo, acessível por navegador em computador ou celular. As capturas de telas estão em NutriVibe/documentacoes/evidencias no repositório. Evidências: https://github.com/huezinho/projeto/tree/main/NutriVibe/documentacoes/ evidencias . A navegação foi verificada em larguras de 360, 768 e 1280 pixels; isso não equivale a uma avaliação completa de usabilidade com participantes externos.
-
-
-
-
-
-
-
-

3. Planejamento do Projeto

O planejamento utiliza o backlog exportado do Jira, com tarefas TRAB-26 a TRAB-57. O arquivo de referência é NutriVibe/documentacoes/historico/backlog-jira.json. As atividades foram organizadas por dependências: estrutura e identidade; perfil e persistência; diário e filtros; resumo e recomendações; reconhecimento e nutrição; testes, documentação e publicação.

A priorização do MVP concentra primeiro o fluxo perfil → registro → resumo. Em seguida entram foto, confirmação nutricional e contato da landing page. Recursos como gamificação completa, contas e sincronização entre dispositivos permanecem fora da entrega atual. O uso de etapas S1, S2 e S3 permite organizar incrementos; não se afirma criação de sprints nativas ou alteração de status no Jira sem registro.

Como organização de acompanhamento, o Kanban deve manter as colunas A Fazer, Em Andamento e Concluído. A Fazer reúne validação externa, ampliação da base de fotos e evoluções de sincronização. Em Andamento corresponde às revisões e validações que a equipe iniciar. Concluído pode receber funcionalidades

cuja implementação e verificação estejam registradas. Essa descrição é uma proposta de organização e não uma captura do estado atual do Jira.

- Paulo é responsável pela frente JavaScript; Davi, por HTML e acompanhamento do Kanban; Victor, por CSS e identidade visual; Marcelo e Akiane, pelas pesquisas. Para organizar a redação, as pesquisas se dividem entre problema/público/requisitos e fontes nutricionais/reconhecimento/limitações, com validação individual dessa distribuição.
- O Scrum orienta a entrega incremental, a revisão do que foi produzido e a adaptação do próximo ciclo. O Kanban torna visíveis responsáveis, dependências e bloqueios. Critérios de conclusão devem incluir funcionamento, revisão e evidência de teste. As próximas etapas priorizam revisão docente, testes com usuários e aparelhos reais e aperfeiçoamento dos limites identificados.
-
-
-
-
-

-
-
-
-

4. Levantamento de Requisitos

Requisitos funcionais: RF01 cadastrar preferências e restrições; RF02 registrar alimentos por data e refeição; RF03 pesquisar e filtrar o catálogo; RF04 tirar ou enviar foto e confirmar/corrigir o resultado; RF05 informar preparo e peso para estimativa nutricional; RF06 consultar resumo diário e sugestões; RF07 remover registros; RF08 exportar, importar e apagar dados locais; RF09 acessar conteúdo educativo; RF10 enviar contato mediante consentimento.

Requisitos não funcionais: interface responsiva e navegação rotulada; validação de entradas; ausência de segredos no frontend; persistência local do diário; tratamento de falhas de armazenamento e rede; rastreabilidade das referências nutricionais; compatibilidade com navegadores capazes de executar JavaScript e WebAssembly. O backend deve validar origem, conteúdo e limites de requisição e utilizar armazenamento persistente para contatos na nuvem.

Regras de negócio: sugestões devem considerar apenas a data selecionada, excluir incompatibilidades e priorizar grupos ausentes. Uma foto não confirma ingredientes ocultos, alergênicos ou peso. O alimento e a referência devem ser

compatíveis e o peso precisa ser válido. Valores nutricionais não informados permanecem ausentes, em vez de serem convertidos em zero. Contato só é gravado com consentimento.

- Restrições: catálogo e referências delimitados, dependência do peso informado e da confirmação humana, reconhecimento sujeito a erros e download inicial volumoso do modelo. O diário não possui login nem sincronização no Neon. Não há comprovação de impacto clínico nem validação comunitária fornecida. As recomendações não substituem avaliação de nutricionista.
-
-
-

5. Desenvolvimento do Projeto

O frontend utiliza HTML semântico, CSS responsivo e módulos JavaScript. app.js conecta eventos, estado e telas; engine.js implementa filtros e recomendações; store.js valida e persiste o diário no localStorage; nutrition.js calcula valores proporcionais; vision-local-entry.js prepara e executa o reconhecimento local, e vision-semantic-policy.js decide se há candidato aceitável.

O backend utiliza Node.js 24. servidor/server.js fornece o handler HTTP e o servidor local; api/index.mjs o adapta à Vercel. As rotas principais são GET /api/status, POST /api/contacts e POST /api/identify. A integração remota com OpenAI ou Vercel AI Gateway permanece opcional; o modo selecionado de reconhecimento usa SigLIP quantizado no navegador, por ONNX Runtime Web e WebAssembly.

O projeto está organizado em html, css, javascript, dados, imagens, servidor, ferramentas, testes e documentacoes. Os scripts de build preparam public e os artefatos do modelo sem publicar documentos internos ou credenciais. Os dados nutricionais são um recorte de 43 referências TACO com preparo, porção-base e proveniência.

- Na nuvem, contatos são persistidos em PostgreSQL/Neon, no banco separado nutrivibe-production. A tabela contacts contém email como chave primária, name obrigatório e consent_at obrigatório. Consultas parametrizadas evitam concatenar entradas em SQL; um contato repetido atualiza nome e consentimento. Localmente, sem conexão PostgreSQL, é utilizado SQLite. O diário alimentar permanece no navegador, separado desse banco.
- Exemplo real de engine.js: export function dailyRecords(records, date) { return records.filter((r) => r.date === date); }. A função seleciona apenas os registros do dia solicitado. Em nutrition.js, a expressão (value * grams) / reference.basisGrams ajusta cada nutriente ao peso confirmado. Em

store.js, localStorage.setItem(KEY, JSON.stringify(validState(state))) valida e serializa o estado antes de persistir.

- Foram implementados perfil, filtros, diário, resumo, recomendações por grupos, backup, educação, cadastro de contatos, reconhecimento com correção manual e cálculo nutricional confirmado. A visão geral passou a mostrar registros reais do dia e atalhos. Permanecem como evolução a sincronização, contas, gamificação completa e validação externa. Código e documentação: https://github.com/huezinho/projeto .
-
-
-
-
-
-
-
-
-
-

6. Testes

Foram executados 25 testes automatizados com node:test. Testes de regras verificam restrições e seletividade, priorização por dia, rejeição de dados inválidos e ausência de relaxamento de filtros. Sua finalidade é evitar recomendações incompatíveis e corrupção do diário durante importação.

Os testes nutricionais verificam proporcionalidade por peso, referências compatíveis e tratamento de valores ausentes. Por exemplo, uma referência de outro alimento deve ser rejeitada, e registros sem informação nutricional não podem ser contados como zero. Os testes de visão verificam classes negativas, margens, vetores inválidos e respostas de provedores. Os de implantação verificam o handler e a persistência compatível com serverless.

O fluxo Chromium verifica perfil, busca, filtros, cadastro e remoção, persistência após recarregar, foto/recusa/confirmação, cálculo nutricional, backup e contatos. Também percorre as cinco telas em 360, 768 e 1280 pixels e verifica transbordamento e navegação móvel. O painel foi verificado com alimento previamente registrado, confirmando sua apresentação e o atalho de retorno ao diário.

- A avaliação de reconhecimento comparou MobileNet, CLIP e SigLIP. O modelo selecionado foi avaliado em 148 imagens; houve 94 acertos do principal alimento/prato em 108 fotos de comida, aproximadamente 87%, e 40 acertos em 50 fotos de comida da coleta adicional, ou 80%. Houve falsos

positivos em plantas/folhas. Esses resultados se referem à amostra, não a uma garantia universal.

- Os resultados motivaram confirmação obrigatória, recusa em baixa similaridade/margem, correção manual e separação entre prato reconhecido e ingredientes. As evidências e resultados estão em documentacoes/avaliacao-imagens e as capturas em documentacoes/evidencias. Os 25 testes e o fluxo Chromium passaram nas verificações registradas. Não foram fornecidos registros de testes de usabilidade com comunidade externa; a verificação técnica não substitui essa etapa.
-
-
-
-
-
-
-
-
-
-

7. Implantação

O sistema foi disponibilizado na Vercel. A landing page é acessada em https://nutrivibe-gamma.vercel.app e o aplicativo em https://nutrivibe-gamma.vercel.app/app . O backend serverless e o Neon atendem o cadastro de contatos. O modo browser-local executa o reconhecimento no aparelho após baixar os artefatos do site.

Para executar localmente, utilizar Node.js 24 e npm. Na raiz do repositório: npm ci para instalar dependências, npm run build para preparar arquivos e modelo, npm test para verificar regras e npm start para iniciar o servidor. A porta padrão é 8080. Abrir http://localhost:8080/html/aplicativo/index.html para o aplicativo; não utilizar file://, pois módulos e recursos dependem do servidor.

No servidor, NUTRIVIBE_VISION_PROVIDER=browser-local seleciona o reconhecimento gratuito. POSTGRES_URL ou DATABASE_URL configura PostgreSQL; sem essas variáveis, o ambiente local utiliza SQLite. Credenciais devem ser configuradas no ambiente e não fazem parte do código entregue. O

navegador precisa de JavaScript, WebAssembly e acesso à rede para o carregamento inicial de aproximadamente 112 MB.

- O build da Vercel usa public como pasta de saída e api/index.mjs como função. O repositório mantém configurações de rotas e cabeçalhos em vercel.json. A disponibilidade depende dos serviços e da conectividade; não foi estabelecido SLA. Em falha de reconhecimento, o catálogo e a correção manual continuam sendo o caminho de registro.
-
-
-
-
-

8. Manutenção / Evoluções Futuras

As próximas evoluções incluem ampliar o catálogo e as referências nutricionais, melhorar o tratamento de pratos compostos, reduzir o custo do primeiro download e avaliar desempenho em celulares de diferentes capacidades. A base de imagens deve crescer com situações reais, fundos variados e controles negativos, sem usar somente exemplos fáceis.

Conta e sincronização entre dispositivos exigirão autenticação, autorização, estratégia de retenção e proteção de dados. Uma eventual persistência remota do diário deve ser projetada explicitamente: o Neon atual guarda contatos, não refeições. Para tráfego maior, é recomendável substituir o limitador em memória por um mecanismo distribuído.

A interface pode evoluir com testes de teclado, contraste, tecnologias assistivas e uso acompanhado por responsáveis. Gamificação completa e novos módulos educativos continuam como possibilidades futuras. Integrações remotas de visão devem ser comparadas por qualidade, custo e privacidade antes de substituir o modo local.

- A avaliação com nutricionista e comunidade externa deve preceder alegações de impacto ou recomendações clínicas. As limitações atuais incluem ausência de identificação garantida de ingredientes/alergênicos/peso, catálogo delimitado, erros de classificação e falta de validação externa documentada.
-
-
-
-
-
-

-

9. Considerações Finais

O NutriVibe transformou uma proposta de educação alimentar em uma solução Web composta por apresentação e aplicativo funcional. O projeto reúne registro, preferências, estimativa nutricional confirmada, sugestões e conteúdo educativo, com código organizado e publicação acessível pela internet.

Os principais resultados técnicos são a separação dos dois sites, o fluxo completo de diário e resumo, o armazenamento local com backup, a integração de contatos ao Neon, a implantação na Vercel e a seleção de reconhecimento visual após comparação documentada. A validação automatizada e no navegador forneceu evidências de funcionamento e também revelou limites que orientaram correções.

O valor pretendido é apoiar responsáveis na organização da rotina e na exploração de variedade alimentar com respeito às preferências. Não se conclui melhora clínica, mudança de hábitos ou impacto comunitário sem dados que sustentem essas afirmações. A continuidade depende de validação com usuários, profissionais e docente.

- Para a formação do grupo, o trabalho articula HTML, CSS, JavaScript, backend, banco de dados, gestão de tarefas e interpretação crítica de resultados. A divisão entre implementação, identidade visual e pesquisa evidencia a importância de integrar contribuições e manter critérios verificáveis. O produto e a documentação permitem revisar escolhas técnicas e planejar a evolução de forma fundamentada.
-
-
-
-
-

-
-
-
-